SMART MUSIC LIBRARY & RECOMMENDATION SYSTEM

Comprehensive Data Structures & Algorithms (DSA) Guidelines & Technical Blueprint

1. EXECUTIVE SUMMARY & ARCHITECTURAL PHILOSOPHY

The Smart Music Library & Graph-Based Recommendation System is an enterprise-grade academic project designed to demonstrate how classical Data Structures and Algorithms (DSA) operate in synergy within a modern full-stack web application. Rather than presenting isolated algorithm exercises, every single data structure serves a concrete, practical, and visible purpose in the user interface and music streaming pipeline.

The system adheres strictly to the blueprint outlined in the project plan, incorporating dual implementations:
1. Pure ANSI C Backend Architecture (located in `backend-c/`) for low-level systems programming and viva demonstrations.
2. High-Performance Node.js Engine (in `dsa/`) powering an interactive REST API, responsive Web UI, audio synthesizer, and real-time DSA visualizer.

2. MASTER DATA STRUCTURES MATRIX

Below is the complete architectural mapping of the seven data structures implemented across the codebase:

- **1. Multi-Way Tree [Hierarchical Music Library]:** Ops: Insert, Hierarchy Search, Path Finding | Time: O(depth) = O(4) = O(1) | Space: O(N)
- **2. Doubly Linked List [Playlist Management]:** Ops: Append, Remove, Reorder, Bidirectional Traversal | Time: Append: O(1), Delete: O(1)* | Space: O(N)
- **3. FIFO Queue [Playback Queue (Up Next)]:** Ops: Enqueue, Dequeue, Play-Next, Peek | Time: Enqueue: O(1), Dequeue: O(1) | Space: O(N)
- **4. LIFO Stack [Playback History (Back Nav)]:** Ops: Push, Pop, Peek | Time: Push: O(1), Pop: O(1) | Space: O(N)
- **5. Weighted Graph [Recommendation Engine]:** Ops: Similarity BFS, 2-Hop Discovery, Ranking | Time: BFS: O(V + E), 1-Hop: O(deg V) | Space: O(V + E)
- **6. Hash Table (Chaining) [Song Exact Lookup by ID]:** Ops: Put, Get, Remove, Search | Time: Avg: O(1), Worst: O(N) | Space: O(M + N)
- **7. Auth Hash Table [5 User Accounts & Auth]:** Ops: Register, Authenticate, Inspect Buckets | Time: Avg: O(1), Worst: O(N) | Space: O(17 + N)

USER AUTHENTICATION & ACCOUNT STORAGE

Data Structure: Hash Table with Polynomial Rolling Hash & Separate Chaining

3. AUTH HASH TABLE & PRE-CONFIGURED USER ACCOUNTS

User accounts are stored in a dedicated Hash Table (`AuthHashTable`) with a prime capacity of M = 17 buckets. Passwords are cryptographically secured using SHA-256 while preserving credential verification via linked bucket chains.

Polynomial Rolling Hash Formula:

For a given username string s of length n, the bucket index is computed as:
H(s) = (" [i = 0 to n-1] s[i] * 37^(n - 1 - i)) mod 17
Where 37 is a prime multiplier minimizing prefix clustering, and 17 is the prime table capacity.

The 5 Pre-Configured Accounts (Ready for 1-Click Evaluator Login):

• **alex_music (Password: MusicPass123!):** Role: Music Enthusiast | Bucket Index: #13 | Pop & Acoustic lover; Divide & Multiply albums

• **sarah_rock (Password: RockOn2026#):** Role: Rock Enthusiast | Bucket Index: #8 | Imagine Dragons & Coldplay high-tempo fan

• **david_jazz (Password: JazzSmooth!45):** Role: Jazz Curator | Bucket Index: #4 | Miles Davis & ambient instrumental enthusiast

• **elena_dsa (Password: GraphAlgo99$):** Role: DSA Evaluator | Bucket Index: #11 | Academic evaluator evaluating time complexities & graphs

• **harsh_admin (Password: SmartMaster77!):** Role: System Administrator | Bucket Index: #2 | Full administrative access and data structure metrics

```
// C Implementation excerpt: include/auth_hash.h & src/auth_hash.c
typedef struct AuthAccount {
    int id;
    char username[64];
    char password[64];
    char name[128];
    char role[64];
    struct AuthAccount* next; // Separate chaining linked list
} AuthAccount;

unsigned int hash_username(const char* username, int capacity) {
    unsigned int hash_val = 0;
    const unsigned int PRIME = 37;
    for (int i = 0; username[i] != '\0'; i++) {
        hash_val = (hash_val * PRIME + (unsigned char)username[i]) % capacity;
    }
    return hash_val;
}
```

CORE DATA STRUCTURES DEEP-DIVE

Hierarchical Tree, Doubly Linked List, Queue, and Stack

4. HIERARCHICAL MUSIC LIBRARY (MULTI-WAY TREE)

• **Structure:** Root -> Genre -> Artist -> Album -> Song

• **Why Chosen:** Music naturally organizes into a taxonomy. A multi-way tree allows intuitive drill-down, filtering by genre or artist, and path reconstruction (e.g., 'Pop -> Ed Sheeran -> Divide -> Perfect').

• **Time Complexity:** Search path: O(depth) = O(4) = O(1); Traversal: O(N) where N is total nodes.

• **C File:** `backend-c/src/library_tree.c` | JS File: `dsa/LibraryTree.js`

5. PLAYLIST MANAGEMENT (DOUBLY LINKED LIST)

• **Structure:** HEAD <-> [Node: Song 1] <-> [Node: Song 2] <-> [Node: Song 3] <-> TAIL

• **Why Chosen:** Playlists require frequent reordering (move up/down) and bidirectional traversal (Previous/Next track). In a Doubly Linked List, unlinking and relinking adjacent nodes takes O(1) pointer operations without shifting elements as an array does.

• **Operations:** append(), prepend(), removeById(), move(up/down), contains() [duplicate detection], getTotalDuration().

• **C File:** `backend-c/src/playlist_list.c` | JS File: `dsa/DoublyLinkedList.js`

6. PLAYBACK QUEUE (FIFO QUEUE)

• **Structure:** FRONT -> [Photograph] -> [Shape of You] -> [Castle on the Hill] <- REAR

• **Why Chosen:** Upcoming music must follow First-In, First-Out (FIFO) semantics. Users can also 'Play Next', which performs an O(1) push at the front of the queue.

• **Time Complexity:** Enqueue: O(1); Dequeue: O(1); Play-Next: O(1); Peek: O(1).

• **C File:** `backend-c/src/playback_queue.c` | JS File: `dsa/PlaybackQueue.js`

7. PLAYBACK HISTORY (LIFO STACK)

• **Structure:** TOP -> [Most Recent Track] -> [Previous Track] -> [Oldest Track]

• **Why Chosen:** When moving forward through music, previous songs are pushed onto the history stack. Clicking the 'Back' button pops the top track in Last-In, First-Out (LIFO) order.

• **Interplay:** Queue and Stack operate in tandem: pressing Next pushes to Stack and dequeues from Queue; pressing Back pops from Stack and prepends old current song back into the Queue.

• **C File:** `backend-c/src/history_stack.c` | JS File: `dsa/HistoryStack.js`

GRAPH-BASED RECOMMENDATIONS & WORKFLOW

Undirected Weighted Graph & Smart Listening Session Integration

8. GRAPH-BASED RECOMMENDATION ENGINE

Songs are modeled as vertices in an Undirected Weighted Graph. An edge is established between two songs if their pairwise metadata similarity score meets or exceeds a configurable threshold (e.g., 30 points).

Similarity Scoring Formula (0 - 100 Points):

- **Same Genre:** +30 points (primary stylistic compatibility)
- **Same Artist:** +25 points (artist catalogue continuity)
- **Similar Mood:** +20 points (emotional alignment: Romantic, Energetic, Relaxing, Sad, Focus)
- **Harmonious Tempo:** +15 points if |BPM1 - BPM2| "d 15; +8 points if |BPM1 - BPM2| "d 30
- **Same Album:** +10 points (thematic album cohesion)

Multi-Hop Graph Discovery Algorithm:

To discover songs that are stylistically connected through intermediaries, the system implements a Breadth-First Search (BFS) multi-hop exploration. A `visited` set prevents cyclic loops and infinite recursion. Candidate recommendations are ranked descending by similarity score, with full explainability tags (e.g., 'Same Genre + Similar Mood').

9. STANDOUT FEATURE: SMART LISTENING SESSION

The Smart Listening Session combines all six data structures into one unified algorithmic workflow:

- **Step 1 (Hash Table):** O(1) lookup of the starting song chosen by the user.
- **Step 2 (Graph Traversal):** Retrieves and ranks 1-hop and 2-hop connected songs via BFS.
- **Step 3 (Filtering):** Excludes songs that do not match the desired mood or that appear in the history stack.
- **Step 4 (Doubly Linked List):** Assembles the filtered songs into a seamless listening sequence totaling the requested duration.
- **Step 5 (FIFO Queue):** Loads the assembled sequence into the playback queue for upcoming playback.
- **Step 6 (LIFO Stack):** Monitors played tracks to update the user's history and ensure no repeat playback.

VIVA VOCE / EVALUATION QUESTIONS & ANSWERS

Comprehensive Academic Q&A for Technical Evaluation

10. FREQUENTLY ASKED VIVA QUESTIONS & MODEL ANSWERS

• **Q1. Why use a Doubly Linked List for playlists instead of a Dynamic Array?:** Playlists involve frequent middle insertions, deletions, and song reordering (Move Up / Down). In an array, moving or deleting an element requires O(N) shifting of subsequent elements. In a Doubly Linked List, once the node is identified, relinking pointers takes O(1) time. Furthermore, prev and next pointers enable natural bidirectional traversal for Next and Previous track controls.

• **Q2. How does the Graph recommendation algorithm prevent infinite loops and duplicates?:** The recommendation engine maintains a `visited` set during BFS traversal. When exploring 2-hop neighbors, any vertex already in the visited set (including the starting song and 1-hop neighbors) is skipped. This guarantees acyclic traversal and prevents duplicate songs in recommendation lists.

• **Q3. What is the difference between your Hash Table lookup and Library Tree search?:** The Hash Table provides O(1) average-time exact retrieval by unique song ID or normalized key. The Tree, in contrast, preserves the categorical hierarchy (Genre -> Artist -> Album -> Song). The Tree is essential for hierarchical browsing and path discovery, whereas the Hash Table is optimal for instant lookups.

• **Q4. How do the Queue and Stack interact during playback?:** They represent forward and backward temporal horizons. The Queue (FIFO) holds the future (upcoming tracks). The Stack (LIFO) holds the past (previously played tracks). When 'Next' is pressed: Current is pushed to History Stack, and Front of Queue is dequeued. When 'Back' is pressed: Top of History Stack is popped to become Current, and old Current is placed back at the Front of Queue.

• **Q5. What is the collision resolution method in your Hash Tables?:** Both SongHashTable and AuthHashTable use Separate Chaining. Each bucket in the table is the head pointer of a singly linked list. When two distinct keys hash to the same bucket index, the new node is prepended to that bucket's chain in O(1) time. In the worst case (all keys colliding), search degrades to O(N), but with prime capacity and our polynomial rolling hash, load factor remains low and search is O(1) on average.

STEP-BY-STEP DEMONSTRATION SCRIPT

7-Step Guided Demonstration for Evaluators and Professors

11. LIVE DEMONSTRATION WALKTHROUGH

• **Step 1: 1-Click Login:** Open http://localhost:5000. Click on any of the 5 pre-configured accounts (e.g. 'elena_dsa'). Note the bucket index (#11) calculated in real-time by the Auth Hash Table.

• **Step 2: Tree Exploration:** Navigate to the 'Library Tree' tab. Expand 'Pop' -> 'Ed Sheeran' -> 'Divide' -> 'Perfect'. Click 'Find Song Path' to observe O(1) hierarchy path resolution.

• **Step 3: Playlist Doubly Linked List:** Open 'Playlists'. Select 'Roadtrip Energy Hits'. Click 'Move Up' on 'Shape of You'. Observe live pointer swap in the UI and the Doubly Linked List Inspector.

• **Step 4: Queue Enqueue & Play Next:** Search for 'Blinding Lights'. Click 'Play Next'. Inspect the Playback Queue tab to verify it inserted at the Front (O(1)) ahead of upcoming tracks.

• **Step 5: Stack Pop & Back Navigation:** Click 'Next' twice on the player bar. Observe the History Stack gaining songs. Click 'Back'. Observe the Stack popping the top song and restoring it as current.

• **Step 6: Graph Recommendations:** Select 'Perfect'. Click 'Discover / Graph'. View the interactive visual graph canvas with weighted similarity links and explainable tags (e.g. 'Same Genre + Same Artist: 55 pts').

• **Step 7: Smart Listening Session:** Choose 'Smart Sessions'. Select 'Perfect', Mood 'Romantic', Duration '30 min', and 'Avoid Recent'. Click 'Generate'. View the execution log combining Hash Table, Graph, Linked List, Queue, and Stack!

12. C SOURCE CODE COMPILATION INSTRUCTIONS

To compile and execute the native ANSI C test suite:
1. Open a terminal in `backend-c/`
2. Run `make` (or `gcc -Wall -Iinclude src/*.c -o smart_music_dsa`)
3. Run `./smart_music_dsa` (or `smart_music_dsa.exe` on Windows)
This outputs an ASCII visualization of all 7 data structures and runs unit verifications.